

```

        makeMove(board, UP);
    } else if (move === LEFT) {
        makeMove(board, RIGHT);
    } else if (move === RIGHT) {
        makeMove(board, LEFT);
    }
}

```

```

function getValidMoves(board, prevMove) {
    // Returns a list of the valid moves to make on this board. If
    // prevMove is provided, do not include the move that would undo it.

    let blankx, blanky;
    [blankx, blanky] = findBlankSpace(board);

    let validMoves = [];
    if (blanky !== SIZE - 1 && prevMove !== DOWN) {
        // Blank space is not on the bottom row.
        validMoves.push(UP);
    }
    if (blankx !== SIZE - 1 && prevMove !== RIGHT) {
        // Blank space is not on the right column.
        validMoves.push(LEFT);
    }
    if (blanky !== 0 && prevMove !== UP) {
        // Blank space is not on the top row.
        validMoves.push(DOWN);
    }
    if (blankx !== 0 && prevMove !== LEFT) {
        // Blank space is not on the left column.
        validMoves.push(RIGHT);
    }
    return validMoves;
}

```

```

function getNewPuzzle() {
    // Get a new puzzle by making random slides from the solved state.
    let board = getNewBoard();
    for (let i = 0; i < DIFFICULTY; i++) {
        let validMoves = getValidMoves(board);
        makeMove(board, validMoves[Math.floor(Math.random() * validMoves.length)]);
    }
}

```